Web Application Security Probe

WASP

Security Assessment Report

Target	http://localhost:8080/sqli_1.php?title=%25&action=search
Scan Date	2026-05-06
Duration	19 seconds
URLs Found	6

Findings Summary

Severity	Count	Risk Level
CRITICAL	1	Immediate action required
HIGH	22	Urgent remediation needed
MEDIUM	0	Fix within 30 days
LOW	0	Fix within 90 days
INFO	0	Informational only

Executive Summary

WASP scanned http://localhost:8080/sqli_1.php?title=%25&action;=search and discovered 23 vulnerability/vulnerabilities, of which 23 are rated HIGH or CRITICAL severity. Immediate remediation is recommended for all high-severity findings. A full review of the application's input validation and output encoding practices should be conducted.

Vulnerability Findings

[1] SQL Injection (Error-based)

Severity	**HIGH**
URL	http://localhost:8080/sqli_1.php?title=%25&action=search
Parameter	title
Payload	'
Evidence	you have an error in your sql syntax
CVSS Score	7.5

Description

SQL Injection occurs when user-supplied input is incorporated into a database query without proper sanitization, allowing attackers to manipulate the query.

Impact

An attacker can read, modify, or delete any data in the database, bypass authentication, and in some cases execute operating system commands.

Remediation

1. Use parameterized queries or prepared statements.
2. Apply input validation and whitelist allowed characters.
3. Use an ORM with built-in protection.
4. Apply the principle of least privilege to DB accounts.
5. Enable a WAF to filter malicious input.

Secure Code Example

```
# VULNERABLE:
query = f"SELECT * FROM users WHERE id = {user_id}"

# SECURE:
query = "SELECT * FROM users WHERE id = %s"
cursor.execute(query, (user_id,))
```

References

• https://owasp.org/www-community/attacks/SQL_Injection
• https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

---

[2] SQL Injection (Form - Error-based)

Severity	**HIGH**
URL	http://localhost:8080/sqli_1.php
Parameter	title
Payload	'
Evidence	you have an error in your sql syntax
CVSS Score	7.5

Description

SQL Injection occurs when user-supplied input is incorporated into a database query without proper sanitization, allowing attackers to manipulate the query.

Impact

An attacker can read, modify, or delete any data in the database, bypass authentication, and in some cases execute operating system commands.

Remediation

1. Use parameterized queries or prepared statements.
2. Apply input validation and whitelist allowed characters.
3. Use an ORM with built-in protection.
4. Apply the principle of least privilege to DB accounts.
5. Enable a WAF to filter malicious input.

Secure Code Example

```
# VULNERABLE:
query = f"SELECT * FROM users WHERE id = {user_id}"

# SECURE:
query = "SELECT * FROM users WHERE id = %s"
cursor.execute(query, (user_id,))
```

References

• https://owasp.org/www-community/attacks/SQL_Injection
• https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

---

[3] Reflected XSS (URL Parameter)

Severity	**HIGH**
URL	http://localhost:8080/sqli_1.php?title=%25&action=search
Parameter	title
Payload	'><script>alert('WASP-XSS')</script>
Evidence	Full payload reflected: '><script>alert('WASP-XSS')</script>
CVSS Score	7.5

Description

Reflected Cross-Site Scripting occurs when user input is immediately included in the page response without proper encoding, allowing script injection.

Impact

An attacker can steal session cookies, perform actions on behalf of the victim, redirect to malicious sites, or capture keystrokes.

Remediation

1. Encode all user-supplied output using context-aware encoding.
2. Implement Content Security Policy (CSP) headers.
3. Use HTTPOnly and Secure flags on session cookies.
4. Validate and sanitize all input server-side.
5. Use a template engine that auto-escapes by default.

Secure Code Example

```
# VULNERABLE (PHP):
echo $_GET['name'];

# SECURE (PHP):
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

References

• https://owasp.org/www-community/attacks/xss/
• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

---

[4] Reflected XSS (URL Parameter)

Severity	**HIGH**
URL	http://localhost:8080/sqli_1.php?title=%25&action=search
Parameter	action
Payload	<img src=x onerror=alert("WASP-XSS")>
Evidence	Dangerous tag/attribute reflected: <img
CVSS Score	7.5

Description

Reflected Cross-Site Scripting occurs when user input is immediately included in the page response without proper encoding, allowing script injection.

Impact

An attacker can steal session cookies, perform actions on behalf of the victim, redirect to malicious sites, or capture keystrokes.

Remediation

1. Encode all user-supplied output using context-aware encoding.
2. Implement Content Security Policy (CSP) headers.
3. Use HTTPOnly and Secure flags on session cookies.
4. Validate and sanitize all input server-side.
5. Use a template engine that auto-escapes by default.

Secure Code Example

```
# VULNERABLE (PHP):
echo $_GET['name'];

# SECURE (PHP):
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

References

• https://owasp.org/www-community/attacks/xss/
• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

---

[5] Reflected XSS (Form Input)

Severity	**HIGH**
URL	http://localhost:8080/sqli_1.php
Parameter	title
Payload	'><script>alert('WASP-XSS')</script>
Evidence	Full payload reflected: '><script>alert('WASP-XSS')</script>
CVSS Score	7.5

Description

Reflected Cross-Site Scripting occurs when user input is immediately included in the page response without proper encoding, allowing script injection.

Impact

An attacker can steal session cookies, perform actions on behalf of the victim, redirect to malicious sites, or capture keystrokes.

Remediation

1. Encode all user-supplied output using context-aware encoding.
2. Implement Content Security Policy (CSP) headers.
3. Use HTTPOnly and Secure flags on session cookies.
4. Validate and sanitize all input server-side.
5. Use a template engine that auto-escapes by default.

Secure Code Example

```
# VULNERABLE (PHP):
echo $_GET['name'];

# SECURE (PHP):
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

References

• https://owasp.org/www-community/attacks/xss/
• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

---

[6] Reflected XSS (Form Input)

Severity	**HIGH**
URL	http://localhost:8080/credits.php
Parameter	security_level
Payload	<img src=x onerror=alert("WASP-XSS")>
Evidence	Dangerous tag/attribute reflected: <img
CVSS Score	7.5

Description

Reflected Cross-Site Scripting occurs when user input is immediately included in the page response without proper encoding, allowing script injection.

Impact

An attacker can steal session cookies, perform actions on behalf of the victim, redirect to malicious sites, or capture keystrokes.

Remediation

1. Encode all user-supplied output using context-aware encoding.
2. Implement Content Security Policy (CSP) headers.
3. Use HTTPOnly and Secure flags on session cookies.
4. Validate and sanitize all input server-side.
5. Use a template engine that auto-escapes by default.

Secure Code Example

```
# VULNERABLE (PHP):
echo $_GET['name'];

# SECURE (PHP):
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

References

• https://owasp.org/www-community/attacks/xss/
• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

---

[7] Reflected XSS (Form Input)

Severity	**HIGH**
URL	http://localhost:8080/portal.php
Parameter	bug
Payload	<img src=x onerror=alert("WASP-XSS")>
Evidence	Dangerous tag/attribute reflected: <img
CVSS Score	7.5

Description

Reflected Cross-Site Scripting occurs when user input is immediately included in the page response without proper encoding, allowing script injection.

Impact

An attacker can steal session cookies, perform actions on behalf of the victim, redirect to malicious sites, or capture keystrokes.

Remediation

1. Encode all user-supplied output using context-aware encoding.
2. Implement Content Security Policy (CSP) headers.
3. Use HTTPOnly and Secure flags on session cookies.
4. Validate and sanitize all input server-side.
5. Use a template engine that auto-escapes by default.

Secure Code Example

```
# VULNERABLE (PHP):
echo $_GET['name'];

# SECURE (PHP):
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

References

• https://owasp.org/www-community/attacks/xss/
• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

---

[8] Reflected XSS (Form Input)

Severity	**HIGH**
URL	http://localhost:8080/password_change.php
Parameter	password_curr, password_new, password_conf
Payload	<img src=x onerror=alert("WASP-XSS")>
Evidence	Dangerous tag/attribute reflected: <img
CVSS Score	7.5

Description

Reflected Cross-Site Scripting occurs when user input is immediately included in the page response without proper encoding, allowing script injection.

Impact

An attacker can steal session cookies, perform actions on behalf of the victim, redirect to malicious sites, or capture keystrokes.

Remediation

1. Encode all user-supplied output using context-aware encoding.
2. Implement Content Security Policy (CSP) headers.

3. Use HTTPOnly and Secure flags on session cookies.
4. Validate and sanitize all input server-side.
5. Use a template engine that auto-escapes by default.

Secure Code Example

```
# VULNERABLE (PHP):
echo $_GET['name'];

# SECURE (PHP):
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

References

• https://owasp.org/www-community/attacks/xss/
• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

---

[9] Reflected XSS (Form Input)

Severity	**HIGH**
URL	http://localhost:8080/user_extra.php
Parameter	login, email, password, password_conf, secret
Payload	<img src=x onerror=alert("WASP-XSS")>
Evidence	Dangerous tag/attribute reflected: <img
CVSS Score	7.5

Description

Reflected Cross-Site Scripting occurs when user input is immediately included in the page response without proper encoding, allowing script injection.

Impact

An attacker can steal session cookies, perform actions on behalf of the victim, redirect to malicious sites, or capture keystrokes.

Remediation

1. Encode all user-supplied output using context-aware encoding.
2. Implement Content Security Policy (CSP) headers.
3. Use HTTPOnly and Secure flags on session cookies.
4. Validate and sanitize all input server-side.
5. Use a template engine that auto-escapes by default.

Secure Code Example

```
# VULNERABLE (PHP):
echo $_GET['name'];

# SECURE (PHP):
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

References

• https://owasp.org/www-community/attacks/xss/
• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

---

[10] Reflected XSS (Form Input)

Severity	**HIGH**
URL	http://localhost:8080/security_level_set.php

Parameter	security_level
Payload	<img src=x onerror=alert("WASP-XSS")>
Evidence	Dangerous tag/attribute reflected: <img
CVSS Score	7.5

Description

Reflected Cross-Site Scripting occurs when user input is immediately included in the page response without proper encoding, allowing script injection.

Impact

An attacker can steal session cookies, perform actions on behalf of the victim, redirect to malicious sites, or capture keystrokes.

Remediation

1. Encode all user-supplied output using context-aware encoding.
2. Implement Content Security Policy (CSP) headers.
3. Use HTTPOnly and Secure flags on session cookies.
4. Validate and sanitize all input server-side.
5. Use a template engine that auto-escapes by default.

Secure Code Example

```
# VULNERABLE (PHP):
echo $_GET['name'];

# SECURE (PHP):
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

References

• https://owasp.org/www-community/attacks/xss/
• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

---

[11] Reflected XSS (Form Input)

Severity	HIGH
URL	http://localhost:8080/sqli_1.php
Parameter	security_level
Payload	<img src=x onerror=alert("WASP-XSS")>
Evidence	Dangerous tag/attribute reflected: <img
CVSS Score	7.5

Description

Reflected Cross-Site Scripting occurs when user input is immediately included in the page response without proper encoding, allowing script injection.

Impact

An attacker can steal session cookies, perform actions on behalf of the victim, redirect to malicious sites, or capture keystrokes.

Remediation

1. Encode all user-supplied output using context-aware encoding.
2. Implement Content Security Policy (CSP) headers.
3. Use HTTPOnly and Secure flags on session cookies.
4. Validate and sanitize all input server-side.

5. Use a template engine that auto-escapes by default.

Secure Code Example

```
# VULNERABLE (PHP):
echo $_GET['name'];

# SECURE (PHP):
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

References

• https://owasp.org/www-community/attacks/xss/
• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

---

[12] Reflected XSS (Form Input)

Severity	**HIGH**
URL	http://localhost:8080/credits.php
Parameter	bug
Payload	<img src=x onerror=alert("WASP-XSS")>
Evidence	Dangerous tag/attribute reflected: <img
CVSS Score	7.5

Description

Reflected Cross-Site Scripting occurs when user input is immediately included in the page response without proper encoding, allowing script injection.

Impact

An attacker can steal session cookies, perform actions on behalf of the victim, redirect to malicious sites, or capture keystrokes.

Remediation

1. Encode all user-supplied output using context-aware encoding.
2. Implement Content Security Policy (CSP) headers.
3. Use HTTPOnly and Secure flags on session cookies.
4. Validate and sanitize all input server-side.
5. Use a template engine that auto-escapes by default.

Secure Code Example

```
# VULNERABLE (PHP):
echo $_GET['name'];

# SECURE (PHP):
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

References

• https://owasp.org/www-community/attacks/xss/
• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

---

[13] Reflected XSS (Form Input)

Severity	**HIGH**
URL	http://localhost:8080/portal.php
Parameter	security_level

Payload	<img src=x onerror=alert("WASP-XSS")>
Evidence	Dangerous tag/attribute reflected: <img
CVSS Score	7.5

Description

Reflected Cross-Site Scripting occurs when user input is immediately included in the page response without proper encoding, allowing script injection.

Impact

An attacker can steal session cookies, perform actions on behalf of the victim, redirect to malicious sites, or capture keystrokes.

Remediation

1. Encode all user-supplied output using context-aware encoding.
2. Implement Content Security Policy (CSP) headers.
3. Use HTTPOnly and Secure flags on session cookies.
4. Validate and sanitize all input server-side.
5. Use a template engine that auto-escapes by default.

Secure Code Example

```
# VULNERABLE (PHP):
echo $_GET['name'];

# SECURE (PHP):
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

References

• https://owasp.org/www-community/attacks/xss/
• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

---

[14] Reflected XSS (Form Input)

Severity	**HIGH**
URL	http://localhost:8080/sqli_1.php
Parameter	bug
Payload	<img src=x onerror=alert("WASP-XSS")>
Evidence	Dangerous tag/attribute reflected: <img
CVSS Score	7.5

Description

Reflected Cross-Site Scripting occurs when user input is immediately included in the page response without proper encoding, allowing script injection.

Impact

An attacker can steal session cookies, perform actions on behalf of the victim, redirect to malicious sites, or capture keystrokes.

Remediation

1. Encode all user-supplied output using context-aware encoding.
2. Implement Content Security Policy (CSP) headers.
3. Use HTTPOnly and Secure flags on session cookies.
4. Validate and sanitize all input server-side.
5. Use a template engine that auto-escapes by default.

Secure Code Example

```
# VULNERABLE (PHP):
echo $_GET['name'];

# SECURE (PHP):
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

References

• https://owasp.org/www-community/attacks/xss/
• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

---

[15] Reflected XSS (Form Input)

Severity	**HIGH**
URL	http://localhost:8080/user_extra.php
Parameter	security_level
Payload	<img src=x onerror=alert("WASP-XSS")>
Evidence	Dangerous tag/attribute reflected: <img
CVSS Score	7.5

Description

Reflected Cross-Site Scripting occurs when user input is immediately included in the page response without proper encoding, allowing script injection.

Impact

An attacker can steal session cookies, perform actions on behalf of the victim, redirect to malicious sites, or capture keystrokes.

Remediation

1. Encode all user-supplied output using context-aware encoding.
2. Implement Content Security Policy (CSP) headers.
3. Use HTTPOnly and Secure flags on session cookies.
4. Validate and sanitize all input server-side.
5. Use a template engine that auto-escapes by default.

Secure Code Example

```
# VULNERABLE (PHP):
echo $_GET['name'];

# SECURE (PHP):
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

References

• https://owasp.org/www-community/attacks/xss/
• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

---

[16] Reflected XSS (Form Input)

Severity	**HIGH**
URL	http://localhost:8080/password_change.php
Parameter	security_level
Payload	<img src=x onerror=alert("WASP-XSS")>
Evidence	Dangerous tag/attribute reflected: <img

CVSS Score	7.5

Description

Reflected Cross-Site Scripting occurs when user input is immediately included in the page response without proper encoding, allowing script injection.

Impact

An attacker can steal session cookies, perform actions on behalf of the victim, redirect to malicious sites, or capture keystrokes.

Remediation

1. Encode all user-supplied output using context-aware encoding.
2. Implement Content Security Policy (CSP) headers.
3. Use HTTPOnly and Secure flags on session cookies.
4. Validate and sanitize all input server-side.
5. Use a template engine that auto-escapes by default.

Secure Code Example

```
# VULNERABLE (PHP):
echo $_GET['name'];

# SECURE (PHP):
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

References

• https://owasp.org/www-community/attacks/xss/
• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

---

[17] Reflected XSS (Form Input)

Severity	**HIGH**
URL	http://localhost:8080/security_level_set.php
Parameter	bug
Payload	<img src=x onerror=alert("WASP-XSS")>
Evidence	Dangerous tag/attribute reflected: <img
CVSS Score	7.5

Description

Reflected Cross-Site Scripting occurs when user input is immediately included in the page response without proper encoding, allowing script injection.

Impact

An attacker can steal session cookies, perform actions on behalf of the victim, redirect to malicious sites, or capture keystrokes.

Remediation

1. Encode all user-supplied output using context-aware encoding.
2. Implement Content Security Policy (CSP) headers.
3. Use HTTPOnly and Secure flags on session cookies.
4. Validate and sanitize all input server-side.
5. Use a template engine that auto-escapes by default.

Secure Code Example

```
# VULNERABLE (PHP):
echo $_GET['name'];
```

```
# SECURE (PHP):
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

References

• https://owasp.org/www-community/attacks/xss/
• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

---

[18] Reflected XSS (Form Input)

Severity	**HIGH**
URL	http://localhost:8080/user_extra.php
Parameter	bug
Payload	<img src=x onerror=alert("WASP-XSS")>
Evidence	Dangerous tag/attribute reflected: <img
CVSS Score	7.5

Description

Reflected Cross-Site Scripting occurs when user input is immediately included in the page response without proper encoding, allowing script injection.

Impact

An attacker can steal session cookies, perform actions on behalf of the victim, redirect to malicious sites, or capture keystrokes.

Remediation

1. Encode all user-supplied output using context-aware encoding.
2. Implement Content Security Policy (CSP) headers.
3. Use HTTPOnly and Secure flags on session cookies.
4. Validate and sanitize all input server-side.
5. Use a template engine that auto-escapes by default.

Secure Code Example

```
# VULNERABLE (PHP):
echo $_GET['name'];

# SECURE (PHP):
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

References

• https://owasp.org/www-community/attacks/xss/
• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

---

[19] Reflected XSS (Form Input)

Severity	**HIGH**
URL	http://localhost:8080/password_change.php
Parameter	bug
Payload	<img src=x onerror=alert("WASP-XSS")>
Evidence	Dangerous tag/attribute reflected: <img
CVSS Score	7.5

Description

Reflected Cross-Site Scripting occurs when user input is immediately included in the page response without proper encoding, allowing script injection.

Impact

An attacker can steal session cookies, perform actions on behalf of the victim, redirect to malicious sites, or capture keystrokes.

Remediation

1. Encode all user-supplied output using context-aware encoding.
2. Implement Content Security Policy (CSP) headers.
3. Use HTTPOnly and Secure flags on session cookies.
4. Validate and sanitize all input server-side.
5. Use a template engine that auto-escapes by default.

Secure Code Example

```
# VULNERABLE (PHP):
echo $_GET['name'];

# SECURE (PHP):
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

References

• https://owasp.org/www-community/attacks/xss/
• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

---

[20] Reflected XSS (Form - Mutated)

Severity	**HIGH**
URL	http://localhost:8080/sqli_1.php
Parameter	title
Payload	'><script>alert('WASP-XSS')</script>
Evidence	Full payload reflected: '><script>alert('WASP-XSS')</script>
CVSS Score	7.5

Description

Reflected Cross-Site Scripting occurs when user input is immediately included in the page response without proper encoding, allowing script injection.

Impact

An attacker can steal session cookies, perform actions on behalf of the victim, redirect to malicious sites, or capture keystrokes.

Remediation

1. Encode all user-supplied output using context-aware encoding.
2. Implement Content Security Policy (CSP) headers.
3. Use HTTPOnly and Secure flags on session cookies.
4. Validate and sanitize all input server-side.
5. Use a template engine that auto-escapes by default.

Secure Code Example

```
# VULNERABLE (PHP):
echo $_GET['name'];

# SECURE (PHP):
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

References

• https://owasp.org/www-community/attacks/xss/
• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

---

[21] Weak/Default Credentials

Severity	**CRITICAL**
URL	http://localhost:8080/user_extra.php
Parameter	login / password
Payload	admin / admin
Evidence	Login succeeded with admin:admin
CVSS Score	9.5

Description

Weak/Default Credentials was detected at http://localhost:8080/user_extra.php.

Impact

Review the finding manually to assess impact.

Remediation

Consult OWASP guidelines for this vulnerability type.

References

• https://owasp.org/www-project-top-ten/

---

[22] CSRF — Missing Token

Severity	**HIGH**
URL	http://localhost:8080/password_change.php
Parameter	N/A
Payload	POST without CSRF token
Evidence	POST form fields: ['password_curr', 'password_new', 'password_conf'] — none matc
CVSS Score	7.5

Description

CSRF — Missing Token was detected at http://localhost:8080/password_change.php.

Impact

Review the finding manually to assess impact.

Remediation

Consult OWASP guidelines for this vulnerability type.

References

• https://owasp.org/www-project-top-ten/

---

[23] CSRF — Missing Token

Severity	**HIGH**
URL	http://localhost:8080/user_extra.php
Parameter	N/A
Payload	POST without CSRF token

Evidence	POST form fields: ['login', 'email', 'password', 'password_conf', 'secret', 'mai
CVSS Score	7.5

Description

CSRF — Missing Token was detected at http://localhost:8080/user_extra.php.

Impact

Review the finding manually to assess impact.

Remediation

Consult OWASP guidelines for this vulnerability type.

References

• https://owasp.org/www-project-top-ten/

Open Ports

Port	State	Service
445	open	SMB
8080	open	HTTP-Alt

Crawled URLs

#	URL
1	http://localhost:8080/sqli_1.php?title=%25&action=search
2	http://localhost:8080/portal.php
3	http://localhost:8080/password_change.php
4	http://localhost:8080/user_extra.php
5	http://localhost:8080/security_level_set.php
6	http://localhost:8080/credits.php